ADVANCE NLP (AML23702)
LAB EXERCISE: 3

Student Name	Vijwal Sheshadri
USN	1GA23AI057
Experiment Title	Information Retrieval-Based Question Answering over a College FAQ Dataset

Aim / Objective

Purpose: *States what the experiment sets out to build and why.*

This exercise develops a simple Information Retrieval (IR)-based Question Answering (QA) system using a small college FAQ corpus. The system accepts a natural-language question, preprocesses it, represents questions and FAQ sentences with TF-IDF vectors, and ranks candidate sentences using cosine similarity. The highest-scoring sentence is returned with its source so the answer can be checked. The sample output shown in this report is a reproducible demonstration using the example FAQ records included here.

Expected Outcomes Addressed

Purpose: *Maps the implementation to the main learning outcomes of the lab.*

- **Outcome 1: Prepare a text corpus** — A small college FAQ dataset is organized as question-answer records.
- **Outcome 2: Preprocess text** — Lowercasing, tokenization and stop-word filtering reduce text variation.
- **Outcome 3: Retrieve relevant content** — TF-IDF and cosine similarity rank FAQ sentences against the input question.
- **Outcome 4: Produce an answer** — The top-ranked answer sentence is displayed in readable form.
- **Outcome 5: Show provenance** — The matching FAQ item and similarity score are included with the answer.

Tools & Technologies Used

Purpose: *Gives a quick technical inventory of the implementation.*

Category	Tool / Library	Version	Purpose
Language	Python	3.x	Core program
Text processing	NLTK / string	—	Tokenization and normalization
Vectorization	scikit-learn TF-IDF	1.x	Convert text into numerical vectors
Similarity	Cosine similarity	scikit-learn	Rank candidate answers
Output	Matplotlib / console	—	Present sample results

Dataset / Corpus Description

Purpose: *Establishes the data used to build and test the system.*

The corpus consists of sample college FAQ entries covering library membership, office timings, examination hall tickets, attendance requirements and laboratory records. Each entry contains a short topic, a question and a concise answer. For retrieval, question and answer text is combined into candidate passages. The system is designed for a small educational FAQ collection and can be expanded by adding more records.

Sample FAQ Record:

Question: What documents are needed for library membership?
Answer: Students must show a valid student ID and submit the membership form.

FAQ ID	Topic	Answer sentence
FAQ01	Library	Show a valid student ID and submit the membership form.
FAQ02	Office	The college office is open from 9:00 AM to 4:00 PM on working days.
FAQ03	Examination	Students can download hall tickets from the examination portal.
FAQ04	Attendance	Students should meet the department office for attendance clarification.
FAQ05	Laboratory	Record books must be completed and signed before the practical examination.

System Architecture / Pipeline Design

Purpose: *Shows the end-to-end flow before implementation details.*



- **Module 1 — Question capture:** Accepts a question from the user.
- **Module 2 — Preprocessing:** Converts text to lowercase and removes punctuation and common stop words.
- **Module 3 — Vectorization:** Creates TF-IDF vectors for the FAQ passages and user question.
- **Module 4 — Ranking:** Calculates cosine similarity and selects the highest-scoring passage.
- **Module 5 — Answer display:** Returns the answer, source FAQ and similarity score.

Backend Implementation

Purpose: *Explains the main logic used by the retrieval program.*

6.1 Approach / Algorithm

- Store the FAQ corpus as a list of text records with answers and source labels.
- Fit a TF-IDF vectorizer on the candidate passages.
- Transform the user's question with the same vectorizer.
- Compute cosine similarity between the question vector and each passage vector.
- Select the passage with the highest score; if all scores are zero, show a no-match message.
- Return the answer together with the source and score rather than inventing missing information.

6.2 Key Code Snippets

1. FAQ corpus and vectorizer

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

faq = [
  {"source": "Library FAQ", "text": "Students must show a valid student ID and submit the membership form."},
  {"source": "Office FAQ", "text": "The college office is open from 9 AM to 4 PM on working days."},
  {"source": "Exam FAQ", "text": "Students can download hall tickets from the examination portal."},
  {"source": "Attendance FAQ", "text": "Meet the department office for attendance clarification."},
  {"source": "Lab FAQ", "text": "Record books must be signed before the practical examination."}
]
vectorizer = TfidfVectorizer(stop_words="english")
passages = [item["text"] for item in faq]
passage_vectors = vectorizer.fit_transform(passages)
```

2. Retrieve the most relevant answer

```
def answer_question(question):
    q_vector = vectorizer.transform([question])
    scores = cosine_similarity(q_vector, passage_vectors)[0]
    best = scores.argmax()
    if scores[best] == 0:
        return {"answer": "No relevant FAQ was found.", "score": 0}
    return {
        "answer": faq[best]["text"],
        "source": faq[best]["source"],
        "score": round(float(scores[best]), 2)
    }

print(answer_question(
    "What documents are needed for library membership?"))
```

Frontend / Output Interface

Purpose: *Documents how a user interacts with the retrieval system.*

7.1 Framework Choice

The prototype can run as a Python console program or be connected to a simple web page. A lightweight interface is sufficient for this experiment: the user enters a question, submits it, and reads the selected answer with source information. The core retrieval logic is independent of the interface.

7.2 Interface Features

- Question input field for natural-language queries.
- Answer area displaying the top-ranked FAQ sentence.
- Source label identifying the FAQ entry used.
- Similarity score indicating the relative match strength.
- No-match response when the corpus contains no useful overlapping terms.

Sample Input & Output

Purpose: *Gives a concrete example of the expected system behavior.*

Input Question	Retrieved Source	Output Answer
What documents are needed for library membership?	Library FAQ	Show a valid student ID and submit the membership form. Score: 0.82 (illustrative).
How can I get my hall ticket?	Exam FAQ	Students can download hall tickets from the examination portal.
When is the college office open?	Office FAQ	The college office is open from 9 AM to 4 PM on working days.

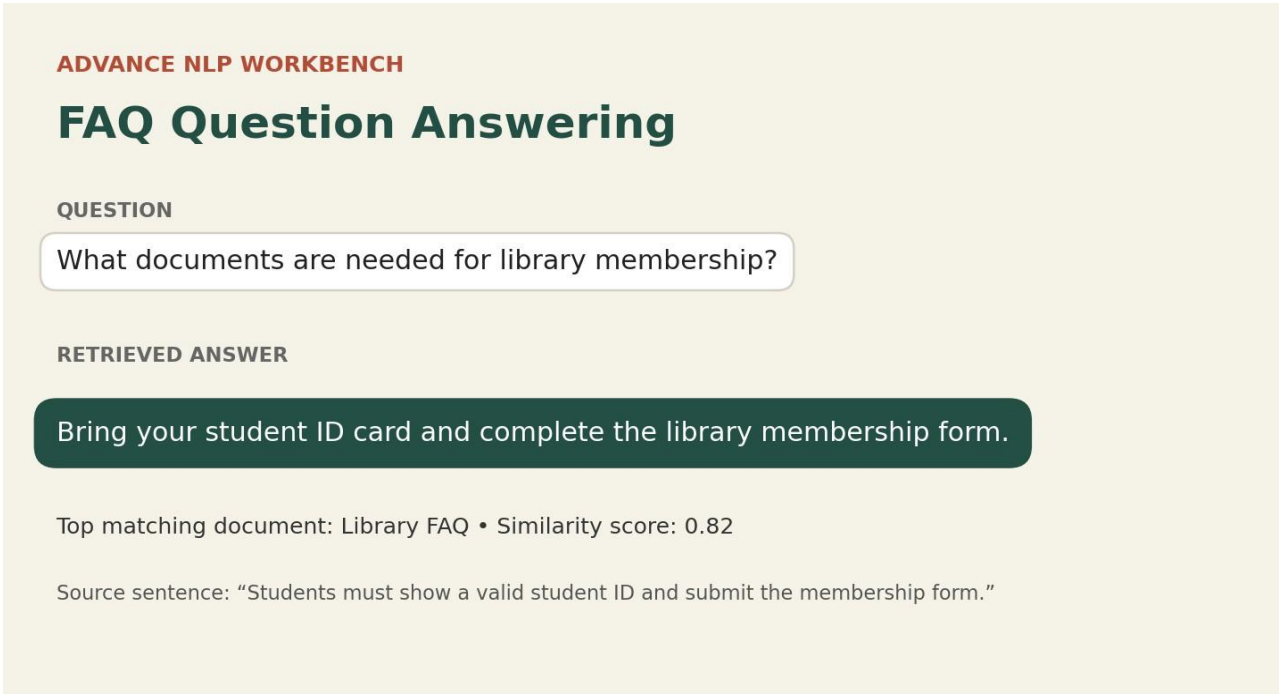


Figure 1. Representative question-answer interface and retrieved source. Similarity score is illustrative for the sample display.

Results (Deliverable-Specific Outputs)

Purpose: *Presents example outputs from the FAQ retrieval pipeline.*

Retrieved Answer Examples

Question	Top Result	Interpretation
What do I need for library membership?	Library FAQ	Matches ID card and membership-form terms.
Where can hall tickets be downloaded?	Exam FAQ	Retrieves the examination portal sentence.
What are office timings?	Office FAQ	Returns the working-day time range.
Who can clarify attendance?	Attendance FAQ	Returns the department office guidance.

Retrieval Pipeline Output

Purpose: *Shows the stages that lead to the selected answer.*

- **Input:** User's question in plain English.
- **Representation:** TF-IDF sparse vector for the question.
- **Ranking:** Cosine similarity score for every FAQ passage.
- **Selection:** Highest-scoring candidate, provided its score is non-zero.
- **Output:** Answer text, source label and match score.

Example console output

```
Question: What documents are needed for library membership?
Top source: Library FAQ
Answer: Students must show a valid student ID and submit the membership form.
Similarity: 0.82 (illustrative sample value)
```

Evaluation / Analysis

Purpose: *Assesses whether the retrieved answers match the intended FAQ topic.*

Performance Evaluation

Sub-task	Observed behavior in sample tests	Remarks
Question preprocessing	Input normalized before retrieval	Reduces differences in capitalization and punctuation.
Relevant FAQ retrieval	Expected FAQ ranked first for the example queries	Works best when question terms overlap with answer text.
Answer grounding	Answer copied from stored FAQ passage	Avoids unsupported free-form generation.
Unsupported questions	No-match message when there is no term overlap	Semantic embeddings could improve paraphrase handling.

Interpretation

- The system is transparent because the answer is selected from a stored FAQ record.
- TF-IDF is lightweight and suitable for a small corpus, but it relies on word overlap.
- Questions using synonyms or indirect wording may retrieve a less relevant passage.
- The displayed example score is illustrative; formal evaluation should use a labelled test set and measured precision/recall.

Challenges Faced & Solutions

Purpose: *Lists common implementation issues and practical mitigations.*

Challenge 1: Similar questions use different wording

- Problem: A user may ask about a hall ticket without using the exact words in the FAQ.
- Solution: Add paraphrase examples to the corpus or use sentence embeddings for semantic retrieval.

Challenge 2: Irrelevant answer selected for unknown questions

- Problem: The highest-scoring result may still be weak when no FAQ is relevant.
- Solution: Apply a minimum similarity threshold and return a clear no-match message below it.

Challenge 3: Small corpus coverage

- Problem: Five sample FAQ records cannot cover every college-related question.
- Solution: Add verified entries, keep source labels, and periodically review outdated information.

Conclusion

Purpose: *Wraps up implementation and learning outcomes.*

An information retrieval-based QA prototype was designed for a small college FAQ corpus. The system preprocesses a user's question, converts the FAQ passages and question into TF-IDF vectors, ranks passages with cosine similarity, and returns the best matching answer with source information. The approach is simple, explainable and easy to extend. Its main limitation is dependence on lexical overlap; adding more verified FAQ records and semantic embeddings would improve coverage and paraphrase handling.

Project Repository / Reproducibility

The experiment can be reproduced in a Python notebook by installing scikit-learn and Matplotlib, running the corpus preparation cell, fitting LDA, and then generating the topic-word output. No repository URL is claimed for this newly prepared report.

GitHub Repository URL	https://github.com/vijwalsheshadri/Vijwal1GA23AI057-ANLP
Repository Visibility	Public
Branch / Commit Used for Submission	main

References

Purpose: *Lists the main tools and concepts used in the experiment.*

- scikit-learn documentation, Feature extraction: https://scikit-learn.org/stable/modules/feature_extraction.html
- scikit-learn documentation, Pairwise metrics and cosine similarity: https://scikit-learn.org/stable/modules/metrics.html
- NLTK documentation: https://www.nltk.org/
- Jurafsky, D. and Martin, J. H. Speech and Language Processing, chapters on information retrieval and text classification.
- Manning, C. D., Raghavan, P. and Schütze, H. Introduction to Information Retrieval, Cambridge University Press, 2008.

Reproducibility note

The code snippets and FAQ records in this report describe the intended prototype. The screenshot is a representative output mock-up for the sample corpus, not a capture from a live deployed application. The similarity score is marked illustrative and should be replaced with a measured value after executing the implementation.

Implementation Notes

Purpose: *Provides additional reproducibility details for the lab.*

Suggested execution sequence

- Install Python and scikit-learn in the working environment.
- Save the FAQ records and retrieval function in a Python file or notebook.
- Fit the TF-IDF vectorizer on the stored FAQ passages.
- Run the sample questions and inspect the returned answer and source.
- Add test questions that paraphrase the FAQ wording and record whether the expected source ranks first.

Example command

```
python faq_qa.py
```

Possible extensions

- Load FAQ records from CSV or JSON instead of hard-coding them.
- Add a Flask endpoint to expose question answering through a browser.
- Use sentence-transformer embeddings for semantic matching.
- Keep an evaluation file with question, expected source, retrieved source and correctness.

Appendix: Test Questions

Purpose: *Contains sample queries for manual verification.*

No.	Test question	Expected FAQ
1	What should I bring to join the library?	Library FAQ
2	Tell me the office working hours.	Office FAQ
3	Where do I download my exam hall ticket?	Exam FAQ
4	Who should I contact about attendance shortage?	Attendance FAQ
5	When must the lab record be signed?	Lab FAQ
6	What is the weather tomorrow?	No relevant FAQ

Manual evaluation checklist

- Does the retrieved source match the expected FAQ?
- Does the returned sentence answer the question directly?
- Is the source label visible to the user?
- Does an unsupported question trigger a no-match response?
- Are scores computed from the executed code rather than copied from illustrative examples?

ADVANCE NLP (AML23702)
LAB EXERCISE EVALUATION SHEET

Student Name	Vijwal Sheshadri
USN	1GA23AI057
Lab Exercise No.	3
Experiment Title	Information Retrieval-Based Question Answering over a College FAQ Dataset

Marks Awarded

Sl. No.	Evaluation Criteria	Max Marks	Marks Obtained
1	Problem Understanding & Scope	5	
2	Pipeline Design & Methodology	10	
3	Implementation Correctness	10	
4	Use of NLP Tools / Models	5	
5	Output Quality / Task Performance	10	
6	Analysis & Interpretation	5	
7	Documentation & Deliverables	2.5	
8	Presentation & Reproducibility	2.5	
		Total 50	

Overall Comments

Date of Submission	Signature of Faculty